

# CSC 4520/6520

# Design & Analysis of Algorithms

---

## CHAPTER 3: BRUTE FORCE ALGORITHMS

Abdullah Bal, PhD

# Objectives

---



## Recap

- Backward Substitution
- $T(n) = T(n - 1) + n$
- $T(n) = n * T(n - 1)$
- $T(n) = T(n / 3) + 1$
- $T(n) = 2T(n - 1) + 1$
- Tower of Hanoi
- Factorial
- Fibonacci Algorithms



## Brute Force



## Selection Sort

## Bubble Sort



## String Matching

## Closest pair problem

# Brute Force

---

- Brute Force Algorithms are exactly what they sound like – straightforward methods of solving a problem that rely on **sheer computing power** and trying every possibility **rather than advanced techniques to improve efficiency**.
- “Just do it!”
- For example, imagine you have a small padlock with 4 digits, each from 0-9. You forgot your combination. How can you find out the correct combination?
- Examples:
  - Computing  $a^n$  ( $a > 0$ ,  $n$  a nonnegative integer)
  - Computing  $n!$
  - Multiplying two matrices
  - Searching for a key of a given value in a list

# Brute Force

---

- The brute-force approach should **not be** overlooked as an **important algorithm design strategy**.
- First, unlike some of the other strategies, brute force is applicable to a very **wide variety of problems**.
- Second, for some important problems—e.g., sorting, searching, matrix multiplication, string matching— the brute-force approach **yields reasonable algorithms** of at least some practical value with no limitation on instance size.
- Third, the expense of designing a more efficient algorithm may be unjustifiable if only **a few instances of a problem** need to be solved and a brute-force algorithm can solve those instances **with acceptable speed**.
- Fourth, even if too inefficient in general, a brute-force algorithm can still be **useful** for **solving small-size instances** of a problem.
- Finally, a brute-force algorithm can serve an important **theoretical or educational purpose** as a yardstick with which to judge more efficient alternatives for solving a problem.

# Selection Sort



# Example 1: Selection Sort Algorithm

- Scan the array to find its smallest element and swap it with the first element.
- Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements.
- Generally, on pass  $i$  ( $0 \leq i \leq n-2$ ), find the smallest element in  $A[i..n-1]$  and swap it with  $A[i]$ :

$A[0] \leq \dots \leq A[i-1] \mid A[i], \dots, A[\text{min}], \dots, A[n-1]$



- After  $n - 1$  passes, the list is sorted.

|    |    |    |    |           |           |           |           |
|----|----|----|----|-----------|-----------|-----------|-----------|
|    | 89 | 45 | 68 | 90        | 29        | 34        | <b>17</b> |
| 17 | 45 | 68 | 90 | <b>29</b> | 34        | 89        |           |
| 17 | 29 | 68 | 90 | 45        | <b>34</b> | 89        |           |
| 17 | 29 | 34 | 90 | <b>45</b> | 68        | 89        |           |
| 17 | 29 | 34 | 45 | 90        | <b>68</b> | 89        |           |
| 17 | 29 | 34 | 45 | 68        | 90        | <b>89</b> |           |
| 17 | 29 | 34 | 45 | 68        | 89        | 90        |           |

# Analysis of Selection Sort

---

**ALGORITHM** *SelectionSort*( $A[0..n - 1]$ )

//Sorts a given array by selection sort

//Input: An array  $A[0..n - 1]$  of orderable elements

//Output: Array  $A[0..n - 1]$  sorted in ascending order

**for**  $i \leftarrow 0$  **to**  $n - 2$  **do**

$min \leftarrow i$

**for**  $j \leftarrow i + 1$  **to**  $n - 1$  **do**

**if**  $A[j] < A[min]$   $min \leftarrow j$

    swap  $A[i]$  and  $A[min]$

# Analysis of Selection Sort

- The input size is given by the number of elements  $n$ ;
- The basic operation is the key comparison  $A[j] < A[\text{min}]$ .
- The number of times it is executed depends only on the array size.

```
ALGORITHM SelectionSort( $A[0..n-1]$ )
//Sorts a given array by selection sort
//Input: An array  $A[0..n-1]$  of orderable elements
//Output: Array  $A[0..n-1]$  sorted in ascending order
for  $i \leftarrow 0$  to  $n-2$  do
     $\text{min} \leftarrow i$ 
    for  $j \leftarrow i+1$  to  $n-1$  do
        if  $A[j] < A[\text{min}]$   $\text{min} \leftarrow j$ 
    swap  $A[i]$  and  $A[\text{min}]$ 
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}.$$

- Thus, selection sort is a  $\Theta(n^2)$  algorithm on all inputs.



# Example of Sorting with Selection Sort

- The action of the algorithm on the list 89, 45, 68, 90, 29, 34, 17 is illustrated as follow:

|    |    |    |    |    |           |           |           |
|----|----|----|----|----|-----------|-----------|-----------|
|    | 89 | 45 | 68 | 90 | 29        | 34        | <b>17</b> |
| 17 |    | 45 | 68 | 90 | <b>29</b> | 34        | 89        |
| 17 | 29 |    | 68 | 90 | 45        | <b>34</b> | 89        |
| 17 | 29 | 34 |    | 90 | <b>45</b> | 68        | 89        |
| 17 | 29 | 34 | 45 |    | 90        | <b>68</b> | 89        |
| 17 | 29 | 34 | 45 | 68 |           | 90        | <b>89</b> |
| 17 | 29 | 34 | 45 | 68 | 89        |           | 90        |

- Each line corresponds to one iteration of the algorithm, i.e., a pass through the list's tail to the right of the vertical bar;
- An element in bold indicates the smallest element found.
- Elements to the left of the vertical bar are in their final positions and are not considered in this and subsequent iterations.

# Bubble Sort Algorithm



## Example 2: Bubble Sort Algorithm

---

- Another brute-force application to the sorting problem is to compare adjacent elements of the list and exchange them if they are out of order.
- By doing it repeatedly, we end up “bubbling up” the largest element to the last position on the list.
- The next pass bubbles up the second largest element, and so on, until after  $n - 1$  passes the list is sorted. Pass  $i$  ( $0 \leq i \leq n - 2$ ) of bubble sort can be represented by the following diagram:

$$A_0, \dots, A_j \overset{?}{\leftrightarrow} A_{j+1}, \dots, A_{n-i-1} \mid A_{n-i} \leq \dots \leq A_{n-1}$$

in their final positions

# Analysis of Bubble Sort Algorithm

---

**ALGORITHM** *BubbleSort*( $A[0..n - 1]$ )

//Sorts a given array by bubble sort

//Input: An array  $A[0..n - 1]$  of orderable elements

//Output: Array  $A[0..n - 1]$  sorted in nondecreasing order

**for**  $i \leftarrow 0$  **to**  $n - 2$  **do**

**for**  $j \leftarrow 0$  **to**  $n - 2 - i$  **do**

**if**  $A[j + 1] < A[j]$  swap  $A[j]$  and  $A[j + 1]$

# Analysis of Bubble Sort Algorithm

- The number of key comparisons for the bubble-sort is the same for all arrays of size  $n$ ;
- It is obtained by a sum that is almost identical to the sum for selection sort:

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2). \end{aligned}$$

**ALGORITHM** *BubbleSort*( $A[0..n-1]$ )  
//Sorts a given array by bubble sort  
//Input: An array  $A[0..n-1]$  of orderable elements  
//Output: Array  $A[0..n-1]$  sorted in nondecreasing order  
**for**  $i \leftarrow 0$  **to**  $n-2$  **do**  
    **for**  $j \leftarrow 0$  **to**  $n-2-i$  **do**  
        **if**  $A[j+1] < A[j]$  **swap**  $A[j]$  and  $A[j+1]$

# Analysis of Bubble Sort Algorithm

- The action of the algorithm on the list 89, 45, 68, 90, 29, 34, 17 is illustrated follow:
- First two passes of bubble sort on the list 89, 45, 68, 90, 29, 34, 17.
- A new line is shown after a swap of two elements is done.
- The elements to the right of the vertical bar are in their final positions and are not considered in subsequent iterations of the algorithm.

|      |                |    |                |    |                |    |                |    |                |    |                |    |
|------|----------------|----|----------------|----|----------------|----|----------------|----|----------------|----|----------------|----|
| 89   | ↔ <sup>?</sup> | 45 |                | 68 |                | 90 |                | 29 |                | 34 |                | 17 |
| 45   |                | 89 | ↔ <sup>?</sup> | 68 |                | 90 |                | 29 |                | 34 |                | 17 |
| 45   |                | 68 |                | 89 | ↔ <sup>?</sup> | 90 | ↔ <sup>?</sup> | 29 |                | 34 |                | 17 |
| 45   |                | 68 |                | 89 |                | 29 |                | 90 | ↔ <sup>?</sup> | 34 |                | 17 |
| 45   |                | 68 |                | 89 |                | 29 |                | 34 |                | 90 | ↔ <sup>?</sup> | 17 |
| 45   |                | 68 |                | 89 |                | 29 |                | 34 |                | 17 |                | 90 |
| 45   | ↔ <sup>?</sup> | 68 | ↔ <sup>?</sup> | 89 | ↔ <sup>?</sup> | 29 |                | 34 |                | 17 |                | 90 |
| 45   |                | 68 |                | 29 |                | 89 | ↔ <sup>?</sup> | 34 |                | 17 |                | 90 |
| 45   |                | 68 |                | 29 |                | 34 |                | 89 | ↔ <sup>?</sup> | 17 |                | 90 |
| 45   |                | 68 |                | 29 |                | 34 |                | 17 |                | 89 |                | 90 |
| etc. |                |    |                |    |                |    |                |    |                |    |                |    |

# Example 3: Brute-Force String Matching

- pattern: a string of  $m$  characters to search for
- text: a (longer) string of  $n$  characters to search in ( $m \leq n$ )
- problem: find a substring in the text that matches the pattern
- We want to find  $i$ —the index of the leftmost character of the first matching substring in the text—such that

$$t_i = p_0 \longrightarrow t_{i+j} = p_j \longrightarrow t_{i+m-1} = p_{m-1}$$

|       |         |                |         |                |         |                |         |           |             |
|-------|---------|----------------|---------|----------------|---------|----------------|---------|-----------|-------------|
| $t_0$ | $\dots$ | $t_i$          | $\dots$ | $t_{i+j}$      | $\dots$ | $t_{i+m-1}$    | $\dots$ | $t_{n-1}$ | text $T$    |
|       |         | $\updownarrow$ |         | $\updownarrow$ |         | $\updownarrow$ |         |           |             |
|       |         | $p_0$          | $\dots$ | $p_j$          | $\dots$ | $p_{m-1}$      |         |           | pattern $P$ |

- If matches other than the first one need to be found, a string-matching algorithm can simply continue working until the entire text is exhausted.

# Brute-Force String Matching

---

- For the string-matching problem:
  - Align the pattern against the first  $m$  characters of the text and start matching the corresponding pairs of characters from left to right until either all the  $m$  pairs of the characters match (then the algorithm can stop) or a mismatching pair is encountered.
  - In the latter case, shift the pattern one position to the right and resume the character comparisons, starting again with the first character of the pattern and its counterpart in the text.
  - The last position in the text that can still be a beginning of a matching substring is  $n - m$  (provided the text positions are indexed from 0 to  $(n - 1)$ ).
  - Beyond that position, there are not enough characters to match the entire pattern; hence, the algorithm need not make any comparisons there.



# Examples of Brute-Force String Matching

---

Pattern: 001011

Text: 10010101101001100101111010

Pattern: happy

Text: It is never too late to have a happy childhood.

# Pseudocode and Efficiency

---

**ALGORITHM** *BruteForceStringMatch*( $T[0..n-1]$ ,  $P[0..m-1]$ )

//Implements brute-force string matching  
//Input: An array  $T[0..n-1]$  of  $n$  characters representing a text and  
// an array  $P[0..m-1]$  of  $m$  characters representing a pattern  
//Output: The index of the first character in the text that starts a  
// matching substring or  $-1$  if the search is unsuccessful

```
for  $i \leftarrow 0$  to  $n - m$  do  
     $j \leftarrow 0$   
    while  $j < m$  and  $P[j] = T[i + j]$  do  
         $j \leftarrow j + 1$   
    if  $j = m$  return  $i$   
return  $-1$ 
```

# Pseudocode and Efficiency

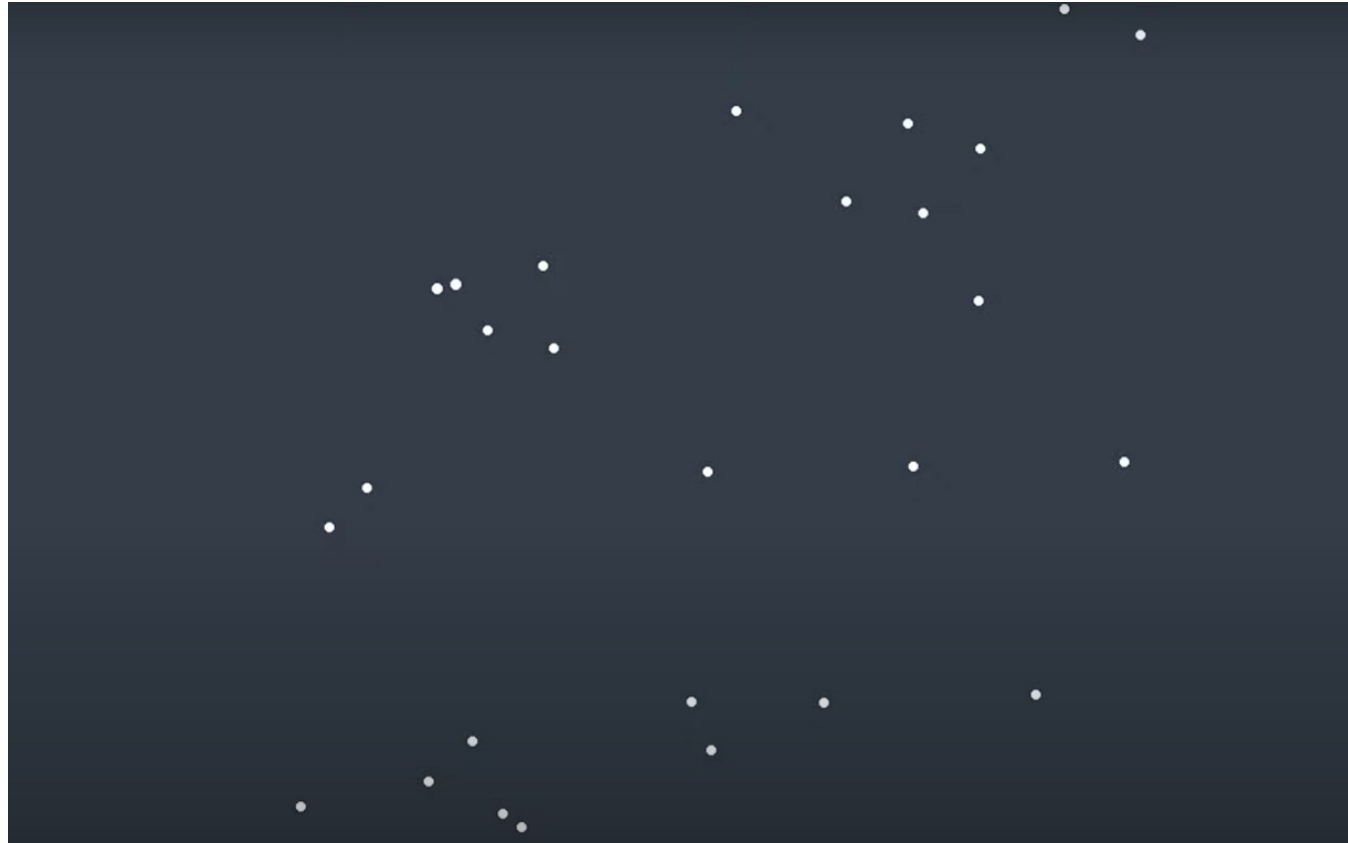
---

- The pattern's characters that are compared with their text counterparts are in bold type.
- Searching in random texts, it has been shown to be linear, i.e.,  $\Theta(n)$ .

N O B O D Y \_ N O T I C E D \_ H I M  
N O N T T T T T T T T  
N O N T O N T O N T O T  
N O N T O N T O N T O T

## Example 4: Closest-Pair Problem

---



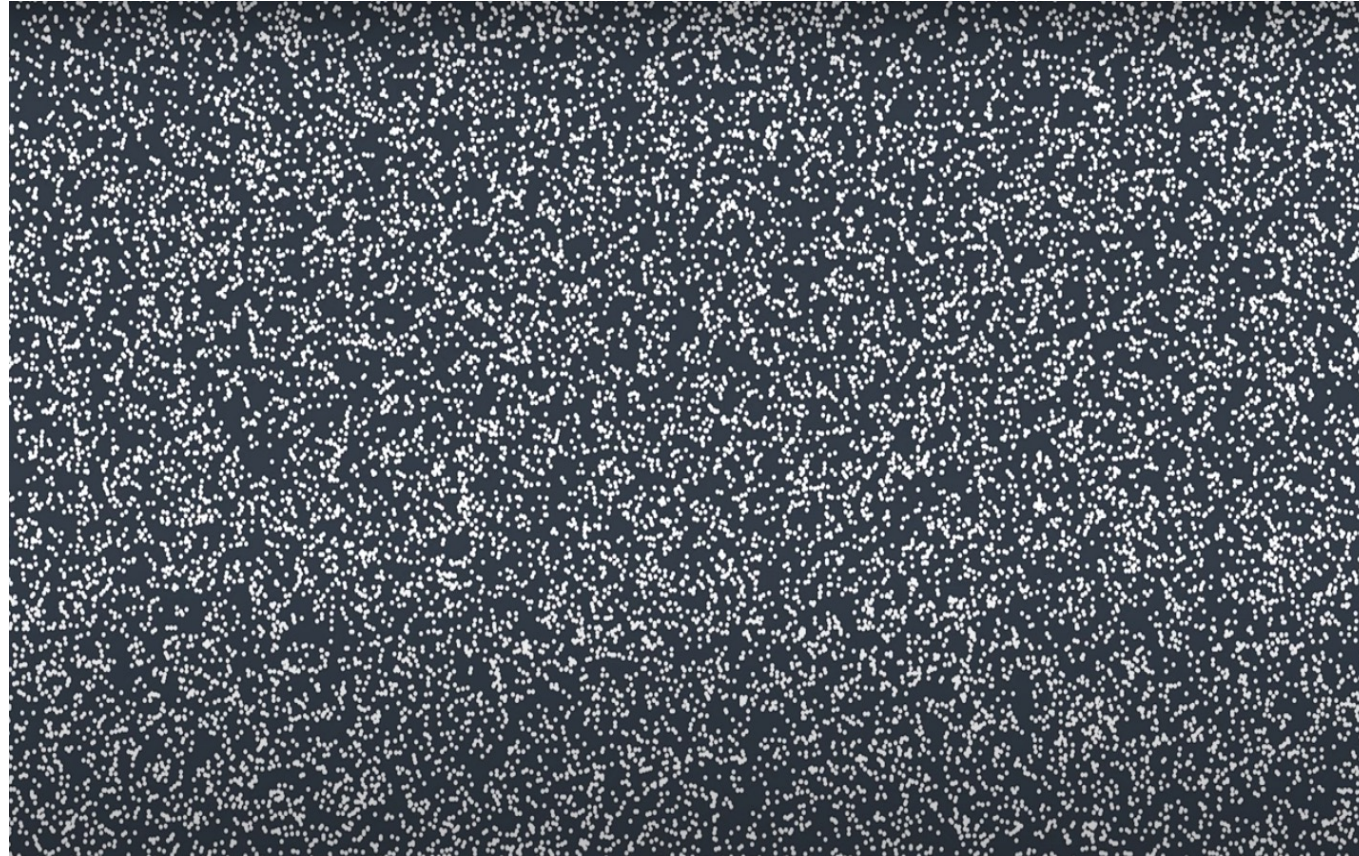
## Example 4: Closest-Pair Problem

---



## Example 4: Closest-Pair Problem

---



# Example 4: Closest-Pair Problem

---

- Find the two closest points in a set of  $n$  points (in the two-dimensional Cartesian plane).
- These problems, arise in two important applied areas: **computational geometry** and **operations research**.
- It is the simplest of a variety of problems in computational geometry that deals with **proximity of points** in the plane or higher-dimensional spaces.
- Points in question can represent such physical objects as airplanes or post offices as well as database records, statistical samples, DNA sequences, and so on.
- An air-traffic controller might be interested in **two closest planes** as the most probable collision candidates.
- A regional postal service manager might need a solution to the closest- pair problem to find candidate **post-office locations** to be closed.



## Example 4: Closest-Pair Problem(Cont.)

---

- One of the important applications of the closest-pair problem is **cluster analysis in statistics**.
- Based on  $n$  data points, **hierarchical cluster analysis** seeks to organize them in a hierarchy of clusters based on some similarity metric.
- For numerical data, this metric is usually the **Euclidean distance**;
- For text and other nonnumerical data, metrics such as the **Hamming distance** are used.
- A bottom-up algorithm begins with each element as a separate cluster and merges them into successively larger clusters by **combining the closest pair of clusters**.



# Euclidian Distance

---

- Distance between two points  $p_i(x_i, y_i)$  and  $p_j(x_j, y_j)$  :

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

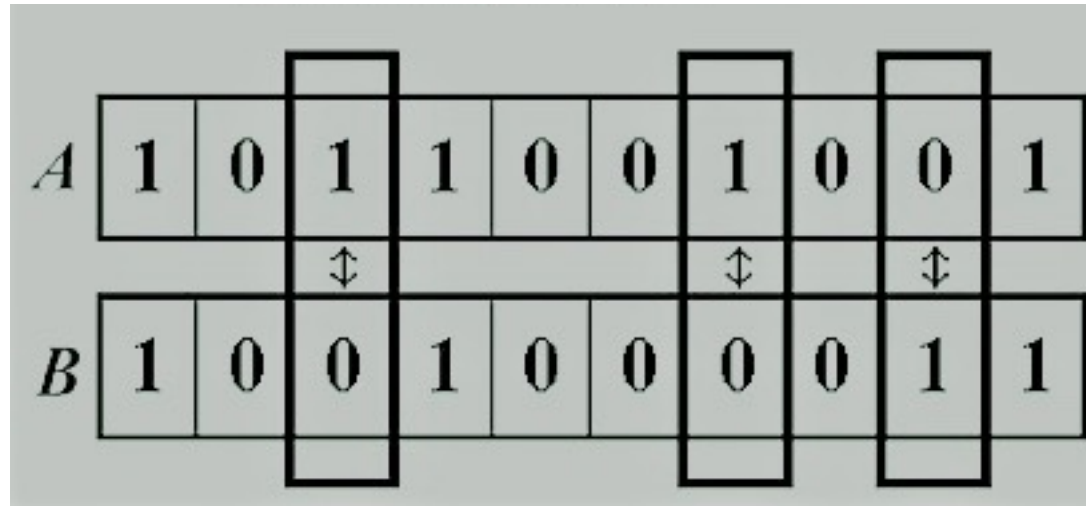
# Hamming Distance (Binary)

---

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$



Hamming distance = 3

## Example 4: Closest-Pair Problem(Cont.)

---

- For simplicity, we consider the **two-dimensional case** of the closest-pair problem.
- We assume that the points in question are specified in a standard fashion by their  $(x, y)$  Cartesian coordinates and that the distance between two points  $p_i(x_i, y_i)$  and  $p_j(x_j, y_j)$  is the standard Euclidean distance

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

- The brute-force approach to solving this problem leads to the following obvious algorithm:
  - **Compute the distance** between each pair of distinct points and find a pair with **the smallest distance**.
  - We do not want to compute the distance between the same pair of points **twice**. To avoid doing so, we consider only the pairs of points  $(p_i, p_j)$  for which  $i < j$ .

# Closest-Pair Brute-Force Algorithm (cont.)

---

**ALGORITHM** *BruteForceClosestPoints(P)*

//Input: A list  $P$  of  $n$  ( $n \geq 2$ ) points  $P_1 = (x_1, y_1), \dots, P_n = (x_n, y_n)$

//Output: Indices  $index1$  and  $index2$  of the closest pair of points

$dmin \leftarrow \infty$

**for**  $i \leftarrow 1$  **to**  $n - 1$  **do**

**for**  $j \leftarrow i + 1$  **to**  $n$  **do**

$d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$  //sqrt is the square root function

**if**  $d < dmin$

$dmin \leftarrow d; index1 \leftarrow i; index2 \leftarrow j$

**return**  $index1, index2$

# Closest-Pair Brute-Force Algorithm (cont.)

- The basic operation of the algorithm will be squaring a number.
- The number of times it will be executed can be computed as follows:

$$\begin{aligned} C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 = 2 \sum_{i=1}^{n-1} (n - i) \\ &= 2[(n - 1) + (n - 2) + \cdots + 1] = (n - 1)n \in \Theta(n^2). \end{aligned}$$

====

## ALGORITHM *BruteForceClosestPoints(P)*

//Input: A list  $P$  of  $n$  ( $n \geq 2$ ) points  $P_1 = (x_1, y_1), \dots, P_n = (x_n, y_n)$

//Output: Indices *index1* and *index2* of the closest pair of points

$d_{min} \leftarrow \infty$

for  $i \leftarrow 1$  to  $n - 1$  do

    for  $j \leftarrow i + 1$  to  $n$  do

$d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$  //sqrt is the square root function

        if  $d < d_{min}$

$d_{min} \leftarrow d$ ;  $\text{index1} \leftarrow i$ ;  $\text{index2} \leftarrow j$

return  $\text{index1}, \text{index2}$

# Brute-Force Strengths and Weaknesses

---

- Strengths

- Wide applicability
- Simplicity
- Yields reasonable algorithms for some important problems  
(e.g., matrix multiplication, sorting, searching, string matching)

# Brute-Force Strengths and Weaknesses

---

- Weaknesses

- Rarely yields efficient algorithms
- Some brute-force algorithms are unacceptably slow
- Not as constructive as some other design techniques

# Summary

---

- Brute Force Algorithms
- Selection Sort
- Bubble Sort
- String Matching
- Closest pair problem



# Supportive Materials

---

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.